THE FOUR PILLARS

OOP is four ideas. A bank account is the example I keep in my pocket.

① Encapsulation

Hide the representation. Only **deposit / withdraw** touch the balance, so the invariant "never negative" lives in one place.

② Abstraction

Show the contract, hide the rest. The ATM offers **withdraw**. You never see the ledger, the SQL, or the vault.

③ Inheritance

SavingsAccount **IS-A** Account. You get reuse and a type you can pass anywhere Account is expected.

④ Polymorphism

One call, many behaviours. The **object's** type picks the code, not the variable's type.

Q01–Q02 · all four in one class

```
public abstract class Account {              // abstraction
  private BigDecimal balance;                  // encapsulation
  public void withdraw(BigDecimal amt) {
    if (amt.compareTo(balance) > 0) throw new InsufficientFunds();
    balance = balance.subtract(amt);            // invariant guarded here
  }
  public abstract BigDecimal interest();        // each type decides
}
class Savings extends Account { ... }         // inheritance

Account a = new Savings();
a.interest();   // polymorphism — runtime type wins
```

say it like this

"Balance is private so the **class** owns its rules. Account is the contract. Savings is a kind of Account. And `interest()` resolves at runtime."

⚠ the interview trap

inheritance is IS-A, not "I needed those fields".
that second one is composition.

up next →

overloading vs overriding, then compile-time vs runtime poly.

#Java #OOP #LearningInPublic

overload vs override

Same name, two completely different machines. One is the compiler. One is the JVM.

Q03 · overloading vs overriding

overloading

```
void print(int n);
void print(String s);
void print(Object o);
// same class, same name,
// different parameter list
```

picked at **compile time** from the static types the compiler can see. return type alone is NOT enough.

overriding

```
class Animal { void speak(){} }
class Dog extends Animal {
  @Override void speak(){}
}
```

picked at **runtime** from the object's real type. same name + same parameters, subclass.

Q04 · compile-time vs runtime polymorphism

`compile-time` — overloading. static binding.

`runtime` — overriding. invokevirtual / invokeinterface.

`visibility` — override can widen, never narrow

`return type` — covariant is allowed (Dog for Animal)

`checked ex` — override may throw fewer / narrower, not new ones

`never override` — static, private, final

overload resolution gotchas

`print(null)` picks the **most specific** type (String over Object) and is ambiguous if two are unrelated. Order the compiler tries: exact → widening → boxing → varargs. So `f(int)` beats `f(Integer)` beats `f(int...)`.

⚠ the interview trap

```
Animal a = new Dog(); a.speak();
```

reference type ≠ the method that runs.

up next →

abstract class vs interface.

#Java #OOP #LearningInPublic

abstract class vs interface

Both say "here is a contract." They are not interchangeable.

Q05 · pick by what you need to share

abstract class

- can hold **state** + constructors
- protected helpers, instance init
- **one** parent only
- IS-A + shared skeleton
- e.g. AbstractList, AbstractMap

interface

- a **capability**: Payable, Repository
- implement many
- no instance fields (constants only)
- Java 8+: default / static methods
- Java 9+: private helper methods

```
interface Notifier {
  void send(String to, String body);          // the contract
  default boolean retryable() { return true; }  // Java 8+
}
abstract class BaseNotifier implements Notifier {
  protected final Clock clock;                  // state — only a class can
  protected BaseNotifier(Clock c){ clock = c; }   // ctor — only a class can
}
```

How I choose

need fields / ctor — abstract class

need many roles — interface

usually both — interface for the API, abstract class for shared guts

diamond — two defaults clash → you must override

constants — interface fields are implicitly public static final

template method — abstract class calls your hook

the sentence that closes it

"An interface is **what** you can do. An abstract class is **how** it is partly done. I start with an interface and add an abstract base only when several implementations genuinely share state."

⚠ the interview trap

"interfaces cannot have method bodies" is 2013.
still true: no instance state on an interface.

up next →

can static methods be overridden? method hiding.

#Java #OOP #LearningInPublic

static & method hiding

If it is static, private, or final — it is not overriding. Full stop.

Q06 · can static methods be overridden?

No. Overriding needs an **instance** to dispatch on. Static methods belong to the **class**, so the compiler binds the call using the **reference type**. What looks like overriding is **hiding**.

```
class Parent { static void hello(){ print("P"); }  void hi(){ print("p"); } }
class Child extends Parent { static void hello(){ print("C"); } @Override void hi(){ print("c"); } }

Parent p = new Child();
p.hello();     // "P"  — static, reference type Parent  (HIDING)
p.hi();        // "c"  — instance, real type Child     (OVERRIDING)
Child.hello(); // "C"  — call statics on the class, not a variable
```

Q07 · what is method hiding?

A subclass declaring a **static** method with the same signature **hides** the parent's. Same rule for **fields** at any access level — fields are never polymorphic, they are resolved by reference type too.

override

instance method · same signature · runtime dispatch · @Override compiles

hide

static method or field · compile-time · reference type wins · @Override fails

field hiding, same trap

`class P { String n = "P"; } class C extends P { String n = "C"; }` → `((P) new C()).n` *is* **"P"**. *Never shadow a field on purpose.*

⚠ the interview trap

@Override on a static method → compile error.
that annotation is the cheapest test you own.

up next →
aggregation vs composition.

#Java #OOP #LearningInPublic

aggregation vs composition

Both are HAS-A. The question is: who dies when the parent dies?

Q08 · ownership of lifetime

aggregation

HAS-A, but the part can live alone.
Department has Professors. Close the department — professors still exist and join another.
In code: you receive it, you do not own its death.

```
class Department {
  private final List<Professor> staff;
  Department(List<Professor> s){
    staff = s;      // handed to me
  }                 // not mine to kill
}
```

composition

HAS-A, and the part cannot outlive the whole.
House has Rooms. Demolish the house — the rooms are gone.
In code: you create it inside and never leak it.

```
class House {
  private final List<Room> rooms;
  House(int n){
    rooms = build(n);   // built here
  }                     // dies with me
}
```

One-liners that land

empty diamond — aggregation (shared lifetime)

filled diamond — composition (owned lifetime)

association — just "knows about", no ownership

dependency — uses it as a method parameter only

Java hint — created in the ctor and never returned? you meant composition

design rule — prefer composition over inheritance for reuse

why interviewers ask

They want to hear **composition + delegation** instead of `extends`. "Stack extends Vector" is the classic mistake — a Stack is not a Vector, it just uses one. Wrap it, expose `push/pop`, and you keep control of the contract.

⚠ the interview trap

Java has no keyword for either one.
It is a design distinction, not a language one.

up next →

SOLID — five rules, one example each.

#Java #OOP #LearningInPublic

SOLID in one page

If you only recite the acronym, you fail. Tie each letter to a class you would split.

Q09 · five reasons a design rots

S **Single Responsibility**

One reason to change. An OrderService that also sends email and writes audit SQL will rot. Split into OrderService, EmailPublisher, AuditLogger.

O **Open / Closed**

Open for extension, closed for modification. A new payment type should be a new class (Strategy), not another `if` in a 400-line switch.

L **Liskov Substitution**

A subtype must honor the parent's contract. Square extends Rectangle then breaks `setWidth` → fails LSP. Test: every test written against the base type must still pass.

I **Interface Segregation**

Many small interfaces beat one fat one. A robot should not be forced to implement `eat()`. In Spring: avoid the 40-method service interface.

D **Dependency Inversion**

Depend on abstractions. OrderService depends on `PaymentClient`, not `StripeHttpClient`. Full page next.

smell → letter

`"and" in the name` — S (UserAndMailService)

`switch grows per feature` — O

`subclass throws Unsupported` — L

`half the methods are no-ops` — I

`service imports an SDK class` — D

`can't unit test without network` — D

⚠ the interview trap

SOLID is not "use Spring." Spring helps you apply D. The other four are still on you.

up next →

DIP vs Dependency Injection — not the same thing.

#Java #SOLID #LearningInPublic

DIP vs Dependency Injection

One is a design rule. The other is a technique (and a Spring feature).

Q10 · Dependency Inversion Principle

High-level policy must not depend on low-level detail. **Both depend on an abstraction**, *and the abstraction is owned by the policy side — not by the SDK.*

```
OrderService  -->  PaymentClient  <--  StripeClient
   (policy)          (abstraction)        (detail)

// before — policy imports the detail
class OrderService { private final StripeHttpClient stripe; }

// after — policy owns the interface, Stripe implements it
class OrderService { private final PaymentClient payments; }
```

Q11 · DIP vs DI

DIP — the principle

A rule about who depends on whom. Invert the arrow so details point at the contract. Plain Java can do this. No framework needed.

DI — the technique

Someone hands you the dependency instead of you calling `new`. Spring, Guice, or just `new OrderService(fake)` in a test. DI is how you fulfill DIP.

`constructor DI` — preferred. final fields, required deps, testable

`field DI` — @Autowired on a field — hides deps, hard to test

`setter DI` — optional / reconfigurable deps

`IoC` — the container calls you; DI is one form of it

⚠ the interview trap

@Autowired does not mean you applied DIP. Inject a concrete StripeClient and you still depend on the detail.

up next →

equals(), hashCode(), and why HashMap cares.

#Java #SOLID #Spring

== vs equals()

Identity vs meaning — then the contract HashMap will punish you for breaking.

Q12 · == vs .equals()

`==` — same reference (for primitives: same value)

`equals()` — logical equality. Object's default IS ==

`Objects.equals` — null-safe, use it

`Integer cache` — -128..127 boxed → == may look true

String literals share the pool, so == can accidentally pass. Never rely on it for content.

Q13–Q14 · the contract

Override `equals` and you **must** override `hashCode`, so equal objects land in the **same bucket**.

`reflexive` — x.equals(x) is true

`symmetric` — x.equals(y) iff y.equals(x)

`transitive` — x=y and y=z ⇒ x=z

`consistent` — don't mutate equality fields in a map

`non-null` — x.equals(null) is false, never NPE

`hashCode` — equal ⇒ hashes MUST match

```
@Override public boolean equals(Object o) {
  if (this == o) return true;
  if (o == null || getClass() != o.getClass()) return false;
  return Objects.equals(email, ((User) o).email);   // stable business key
}
@Override public int hashCode() { return Objects.hash(email); }
```

⚠ the interview trap

unequal objects MAY share a hash → collision, fine.
equal objects MUST share a hash → that is the law.

up next →

how HashMap actually uses these two methods.

#Java #equals #hashCode

how HashMap uses hashCode + equals

hashCode finds the hallway. equals finds the door.

Q15 · put / get in one breath

```
// 1. spread — mix high bits down so they affect the index
h = key.hashCode(); h ^= (h >>> 16);
// 2. bucket — cheap modulo, works because capacity is a power of 2
i = (capacity - 1) & h;
// 3. walk that bucket (list or tree)
// 4. equals decides:  same key -> replace value  |  different -> append
```

get follows the identical path. If either step disagrees with put, the entry is invisible.

Q16 · same hash, two objects

A **collision**. Both live in one bucket; HashMap walks it and calls `equals` to tell them apart. Perfectly legal — just slower. A constant `hashCode() { return 1; }` is "correct" and turns the map into a list.

Q17 · equal objects, different hashes

You broke the contract. `put(a,v)` lands in bucket H1; `get(b)` where `a.equals(b)` looks in H2 → **miss**. HashSet happily keeps "duplicates". The map looks haunted.

equals, no hashCode

logical twins in different buckets

hashCode, no equals

pointless — lookup still calls equals

mutable key

mutate after put → orphaned entry, leak

good keys

immutable, small, stable: String, Long, UUID, enum, record. Bad keys: JPA entities keyed on a null `@Id`, or anything with a mutable list inside.

⚠ the interview trap

never mutate a field used in hashCode after the object is already a map key.

up next →

HashMap internals: buckets, treeify, load factor, resize.

#Java #HashMap

HashMap internals

Array of bins. Index from the hash. Collisions chain, then sometimes become a tree.

18 How does it work internally?

A `Node[] table` of buckets, capacity always a power of 2, so `index = (capacity-1) & hash`. Empty bin → place the node. Occupied → walk the chain (or tree), `equals` to replace or append.

19 How are collisions handled?

Same index, different keys → singly linked list of Nodes. Java 8+: a hot bin converts to a red-black tree of TreeNodes, so the worst case is O(log n) instead of O(n).

20 When does a bucket become a tree?

At 8 nodes in that bin and table capacity ≥ 64. Below 64 it resizes instead, because spreading is cheaper than treeing. Reverts to a list at 6. Tree order needs Comparable keys, else it falls back to identity ordering.

21 What is the load factor?

Default 0.75 — the space/collision trade-off. `threshold = capacity × loadFactor`, so a default map resizes at 12 entries. Lower it for speed, raise it to save memory.

22 What happens on resize?

Capacity doubles. Java 8 does not rehash from scratch; it splits each bin into lo/hi using one extra hash bit, preserving relative order. Resizing is O(n) and allocates a new table — so pre-size it.

constants worth memorising

`DEFAULT_CAPACITY` — 16

`LOAD_FACTOR` — 0.75

`TREEIFY` — 8

`UNTREEIFY` — 6

`MIN_TREEIFY_CAP` — 64

`pre-size` — new HashMap<>((int)(n/0.75)+1)

⚠ the interview trap

treeify is not "always at 8" — capacity must be ≥ 64.
and HashMap is still not thread-safe.

up next →

null keys, ConcurrentHashMap, Hashtable.

#Java #HashMap

nulls & ConcurrentHashMap

One null key in HashMap. Zero nulls in CHM. Hashtable is a museum piece.

23 Why one null key in HashMap?

Null's hash is treated as 0, so it lives in bucket 0 on a special path. Only one, because keys are unique. Many null values are allowed. Not a feature to design around.

24 Why does CHM forbid nulls?

`get` returning null would be ambiguous: key absent, or key mapped to null? On one thread you can disambiguate with `containsKey`; under concurrency that check is already stale. Doug Lea chose to ban it.

25–26 HashMap vs CHM · how CHM is safe

HashMap takes no locks at all. CHM (Java 8+): empty bin → CAS; occupied bin → synchronized on that bin's first node, never the whole map. Reads are lock-free via volatile fields. Use `compute` / `merge` / `putIfAbsent` for atomic read-modify-write.

27 Java 7 vs Java 8+ CHM

Java 7: 16 segments, one ReentrantLock each — concurrency capped at the segment count. Java 8: per-bin CAS / node locks plus trees for hot bins. Finer granularity, less memory, better under contention.

28 HashMap vs Hashtable vs CHM

HashMap — fast, unsynchronized, one null key. Hashtable — every method synchronized, no nulls, Java 1.0 leftover. CHM — the concurrent map you actually use. Never "fix" HashMap by switching to Hashtable.

atomic, not just thread-safe

`if (!map.containsKey(k)) map.put(k, v);` races even on CHM. Write `map.computeIfAbsent(k, this::load)` or `map.merge(k, 1L, Long::sum)`. Also: `size()` is a striped estimate.

⚠ the interview trap

Collections.synchronizedMap is one coarse lock —
and check-then-act still races through it.

up next →

ArrayList vs LinkedList vs Vector.

#Java #Collections

ArrayList vs LinkedList

Random access vs node pointers. Vector is ArrayList wearing a global lock.

Q29 · the real difference

ArrayList

One contiguous `Object[]`. Default capacity 10, grows ~1.5×. CPU-cache friendly, so it wins far more often than the Big-O suggests. The default List.

LinkedList

Doubly linked nodes, each with prev/next overhead. No cache locality. Also a `Deque`, which is the only reason to reach for it.

cost that matters

`get(i)` — ArrayList O(1) · LinkedList O(n)

`add(0, e)` — O(n) shift · O(1)

`remove via iterator` — O(n) · O(1)

`add(e) at end` — amortised O(1) · O(1)

`add(i, e)` — O(n) · O(n) — must walk to i first

`memory` — array slack · ~2 refs per element

Q30 · when do I pick LinkedList?

Almost never as a List. Only when you constantly add/remove at **both ends** — and even then `ArrayDeque` is usually faster. "Lots of middle inserts" is a myth: `list.add(i, x)` is still O(n) because it must walk to `i`.

Q31 · ArrayList vs Vector

Vector synchronizes **every** method on one lock and doubles (2×) on growth. Java 1.0 leftover. Use ArrayList; if you need threads use `CopyOnWriteArrayList` (read-heavy) or a concurrent collection. Per-method locks don't make iteration safe anyway.

also know

`fail-fast` — modCount check → ConcurrentModificationException

`stack / queue` — ArrayDeque, never the legacy Stack

`safe removal` — iterator.remove() or removeIf()

`ensureCapacity` — pre-size if you know n

⚠ the interview trap

"LinkedList is O(1) insert" — only if you already
hold the node. By index it is still O(n).

up next →

HashSet / LinkedHashSet / TreeSet and the Map cousins.

#Java #Collections

Set & Map flavors

Hash = speed. Linked = order. Tree = sorted. Pick one job.

Q32 · HashSet vs LinkedHashSet vs TreeSet

`HashSet` — a HashMap with dummy values. unordered, O(1), one null

`LinkedHashSet` — O(1) + insertion-order iteration

`TreeSet` — red-black tree, O(log n), sorted, no null

`EnumSet` — bit vector. use it for enums, always

Q33 · HashMap vs LinkedHashMap vs TreeMap

`HashMap` — unordered buckets. the default. not thread-safe

`LinkedHashMap` — insertion or access order → LRU

`TreeMap` — sorted keys, O(log n), NavigableMap

`TreeMap extras` — firstKey, headMap, subMap, ceilingKey

```
// access-order LinkedHashMap = a 10-line LRU cache
Map<K,V> lru = new LinkedHashMap<>(16, 0.75f, true) {
  @Override protected boolean removeEldestEntry(Entry<K,V> e) {
    return size() > 1000;
  }
};
```

declare the interface, not the class

`Map<String,User> m = new HashMap<>();` — then you can swap in LinkedHashMap for ordering, or TreeMap for sorting, without touching callers.

⚠ the interview trap

TreeMap keys must be mutually comparable and consistent with equals — else the map quietly lies.

up next →

how to write an immutable class.

#Java #Collections

immutable classes

Once built, it cannot change. That is why String is a safe HashMap key.

Q34 · the recipe

1. class `final` · 2. fields `private final` · 3. no setters · 4. set everything in the ctor · 5. **defensive copy** mutable things on the way in and out · 6. never leak `this` from the constructor.

```
public final class Money {
  private final BigDecimal amount;
  private final List<String> tags;
  public Money(BigDecimal a, List<String> t) {
    this.amount = a;
    this.tags = List.copyOf(t);          // copy IN — caller keeps the original
  }
  public List<String> tags(){ return tags; }  // already unmodifiable
  public Money plus(BigDecimal d){          // "mutation" returns a new object
    return new Money(amount.add(d), tags);
  }
}
```

Q35 · why generally final?

A subclass could add setters, or override a getter to return something different each call, and **break the promise** callers rely on. `final` shuts that door — exactly why `String` is final. Alternative: private ctor + static factory.

Q36 · defensive copy

Lists, arrays, `Date`, and other mutable fields are shared references. Without copying, the caller keeps a handle to your internals and can mutate "your" state later. `List.copyOf`, `Arrays.copyOf`, `new Date(d.getTime())`, or migrate to `Instant` and `record`.

what immutability buys

thread-safe with zero locks · safe HashMap keys · cacheable hash · free to share and publish · no defensive copies elsewhere.

⚠ the interview trap

`private final List<X> xs;` is not immutable if you stored the caller's ArrayList as-is.

up next →

final, transient, @Transient, finalize().

#Java #Immutability

final · transient · finalize

Three words that look related. They are not.

37 What is final used for?

variable — cannot reassign (the object can still mutate). method — cannot override. class — cannot extend. Blank finals must be assigned in every constructor. "Effectively final" is what a lambda or anonymous class is allowed to capture.

38 What is transient?

Java serialization skips that field. Use it for caches, passwords, sockets, derived values. After deserialization it holds the default (0 / null), so recompute in `readObject` if you need it. Irrelevant for Jackson — that uses `@JsonIgnore`.

39 transient vs JPA @Transient?

Two different worlds. `transient` = "skip during Java serialization". `@Transient` = "this is not a database column". A field can be one, both, or neither. Don't answer one when asked the other.

40 final vs finally vs finalize()

final — a modifier (see Q37). finally — the block that always runs once try is entered, even after `return`. finalize() — an Object hook the GC might call; deprecated since Java 9, unpredictable, may never run. Never write a new one.

what to say instead of finalize()

```
try (var in = Files.newInputStream(path)) {   // closes in reverse order
  return read(in);
}                                             // close() exceptions are suppressed
// need more? java.lang.ref.Cleaner or PhantomReference. never finalize().
```

the finally trap

A `return` or `throw` inside `finally` **overwrites** whatever the try block was returning — the original exception is lost. Never return from finally.

⚠ the interview trap

final object ≠ immutable object.

`final List<X> xs = new ArrayList<>(); xs.add(e);`

compiles.

series done →

next notebook: String, JVM memory, exceptions.

#Java #LearningInPublic